Listing 5.6 `MovieAPI` object

```
interface MovieAPI {
    @GET("/movies")
    suspend fun getMovies(): List<Movie>
}

data class MoviesDto(
  val response: List<Movie>
)

data class Movie(
    val id: Int,
    val name: String,
    val year: Int,
)
```

The interface contains the API request, annotated with `retrofit2.http` request method `GET`, as shown in listing 5.6. The `getMovies` function returns a response with type `MoviesDTO`. The `MoviesDTO` data object returns the response list of the `Movie` object type. The `Movie` data object contains the ID, name, and year of the movie. This outlines how the mobile app will deserialize the JSON response from the movies API. Then, it can be passed to the presentation layer for display within the mobile app. The test hooks into the client, which exposes the API requests.

Listing 5.7 Movie client

```
fun MovieClient(url: String = BASE_URL): MovieAPI {     #1
     return Retrofit.Builder()
     .baseUrl(url)
     .addConverterFactory(GsonConverterFactory.create())
     .build()
     .create(MovieAPI::class.java)
}
```

#1 BASE_URL is an environment variable.

The movie client has been abstracted from the Dagger provider method binding (read more about Dagger basics at https://mng.bz/vJVM), which exposes an implementation of the API client. This has been abstracted so the URL can be passed as an argument to the function, which will become clear later when setting up the Pact mock server for the consumer tests. The movie client function returns the `MovieAPI` interface. Within the function, the retrofit builder defines the base URL and builds the instance of the `MovieAPI` client.